Real-time Computer Graphics – Miniproject / Exercise portfolio

ABSTRACT

This Pictorial template describes the formatting requirements for Pictorial submissions to SIGCHI conference proceedings and offers recommendations on writing for the worldwide SIGCHI readership. Please review this document even if you have submitted to the Pictorials track before, as some format details may have changed relative to previous years. Abstracts should be about and not longer than 150 words and are required.

Keywords

Authors' choice; of terms; separated; by semicolons.

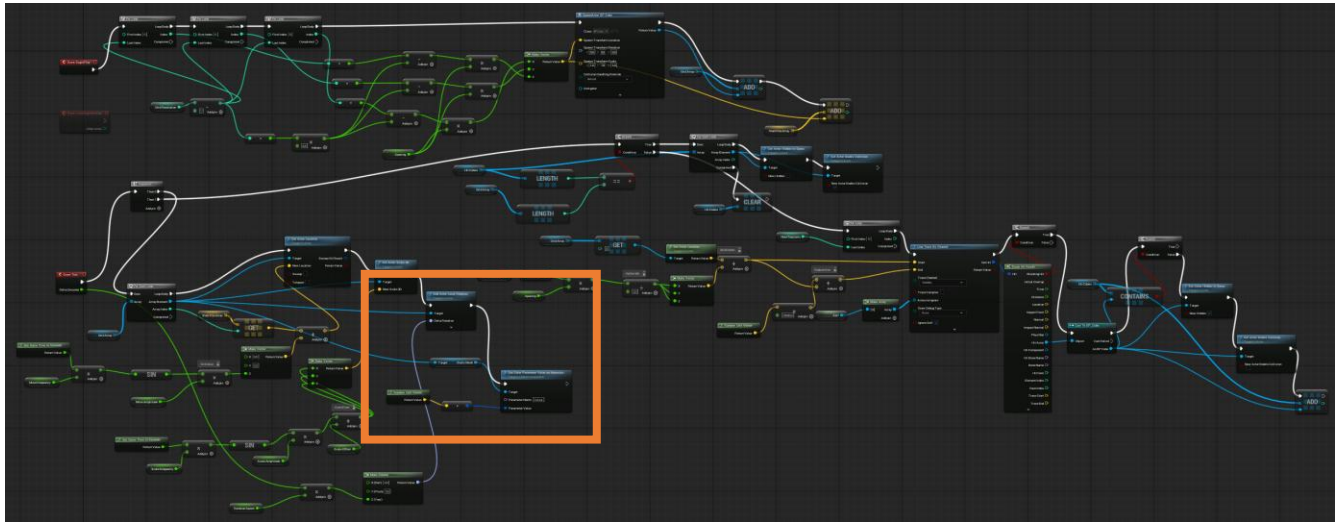
Exercise 1: Intro to Unreal

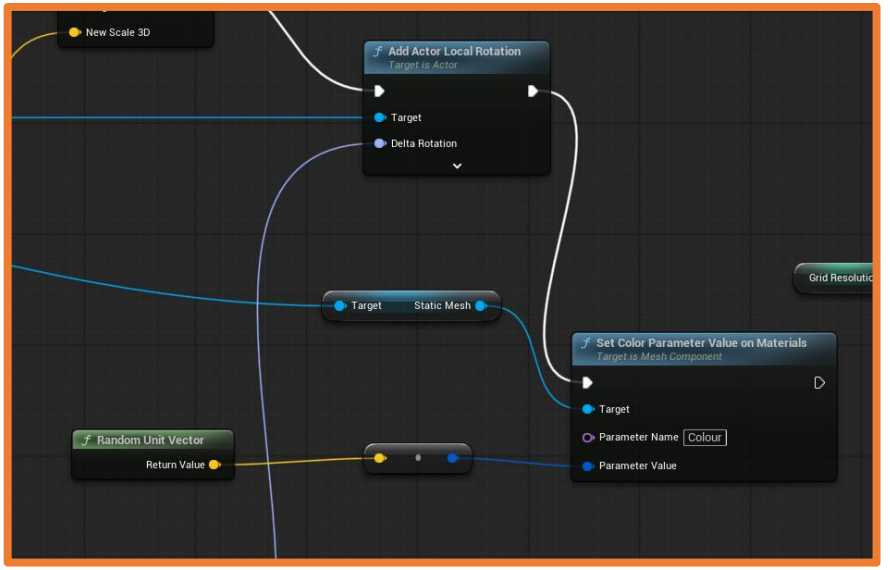
This exercise functioned as an introduction to Unity or Unreal, and given my lack of training in the latter, I chose to work with that as a learning experience.

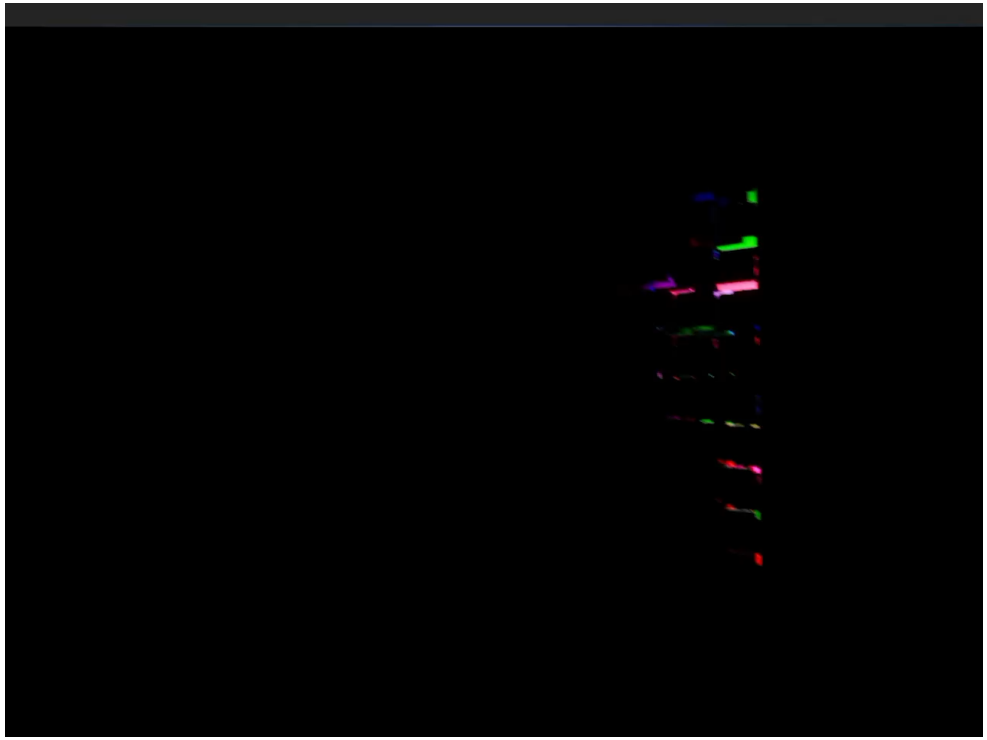
I followed the provided guide to,

1. Instantiate and register a grid of cubes at startup
2. Animate their rotation, position, and size per frame
3. Shoot raycasts from the center of the grid to disable cubes
4. Animate a light source's colour and position

For my own addition, I added a small step to the cube animation where each would randomly change the colour of their material per frame. This required adding a property to the material. For the preview below, I also reduced the amount of raycasts per loop and increased the light's intensity.





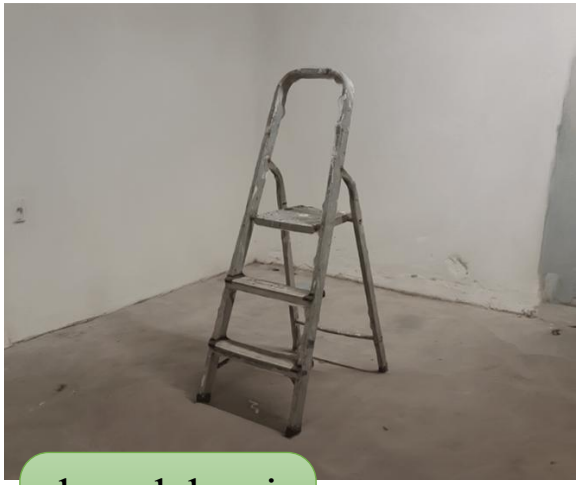


Exercise 2: Intro to Image-based Modelling and Rendering

In this exercise, I tested different tools for image-based modelling, some employing gaussian splatting, while others relied on more classical photogrammetry techniques.

Lumalabs.ai and Postshot created complete scenes with gaussian splatting, either from a provided series of photographs, or an auto-generated series from an input video. These were visually appealing when around the recorded viewing angles, but the illusion broke when focusing on anything besides the primary subject. Lumalabs offers no way of exporting, and getting a model from PostShot involves a lot of tedious filtering, so I didn't attempt this, though its fidelity of the subject was impressive.

Meshroom similarly attempted to construct the surrounding environment with no options to filter for area before running its slow mesh generation.(what method`?) Unlike the previous ones, however, it was capable of producing an exportable mesh and texture, though the texture up-close was quite low fidelity.



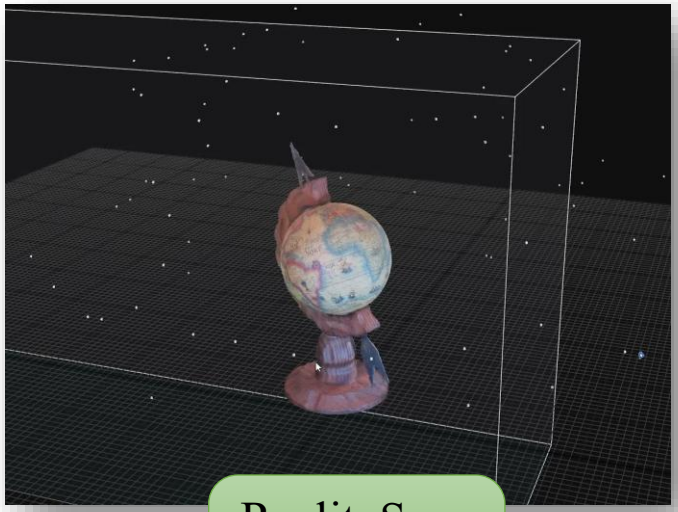
RealityScan proved the best of the options in my experience, at least when it came to recording a physical object and turning it into a textured 3D mesh to use in computer graphics. Here, I created two models of high enough quality to use, an ebony mask and a small globe.

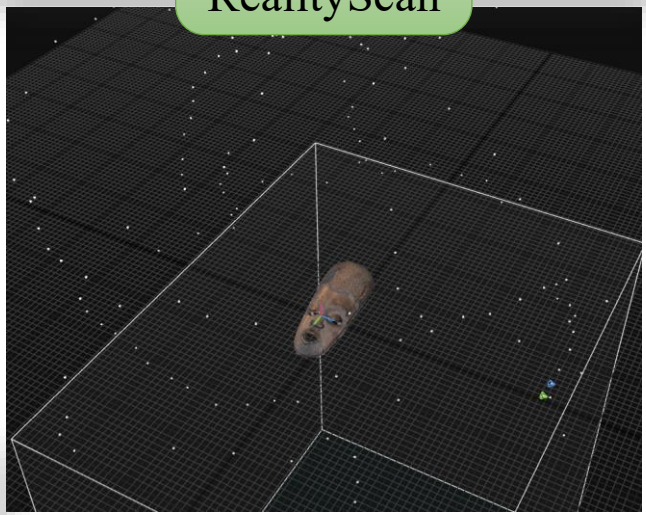
These objects were appropriate challenges because they had a few gaps and self-occlusions, but were fully opaque and largely non-reflective, ensuring consistent image features across different capturing angles.

Finalizing their models involved a lot of filtering, primarily cutting by box and careful edge cleaning, with some gaps having to be repaired particularly around the eyes and mouth of the ebony mask.







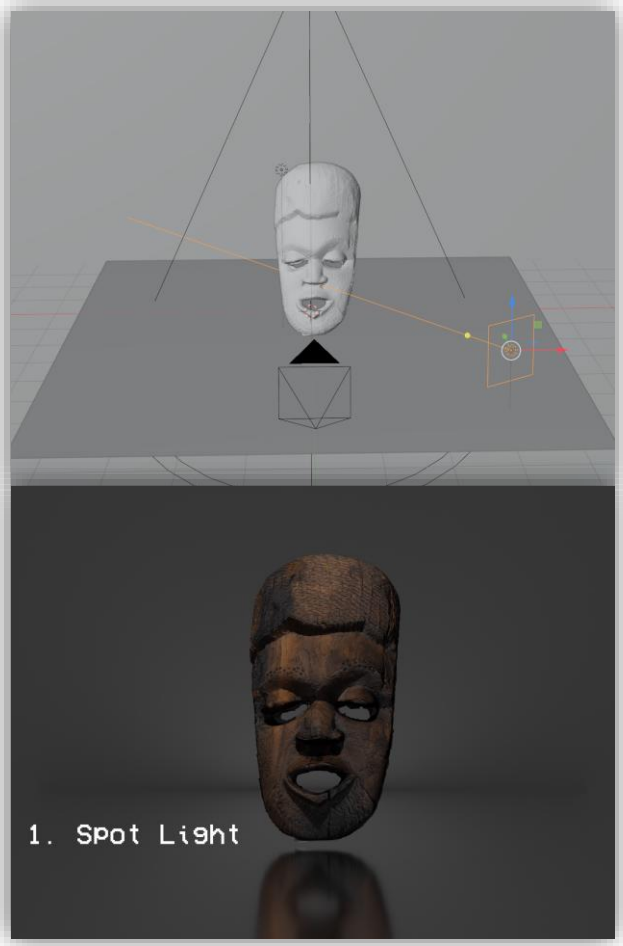


Exercise 3: Fundamental Concepts in Computer Graphics Lighting

-spot from above creates highlights on up-facing surfaces and dark spots beneath them

-area light fills in dark spots created by spot's angle creates a soft shadow on the backdrop

-point light creates an accent from the opposite direction





Exercise 4: Aspects of Camera Parameters in Computer graphics

This exercise was intended to demonstrate different aspects of camera parameters in Unreal or Unity. I chose Unreal and created a short animated clip including two effects, depth of field and dynamic lighting.

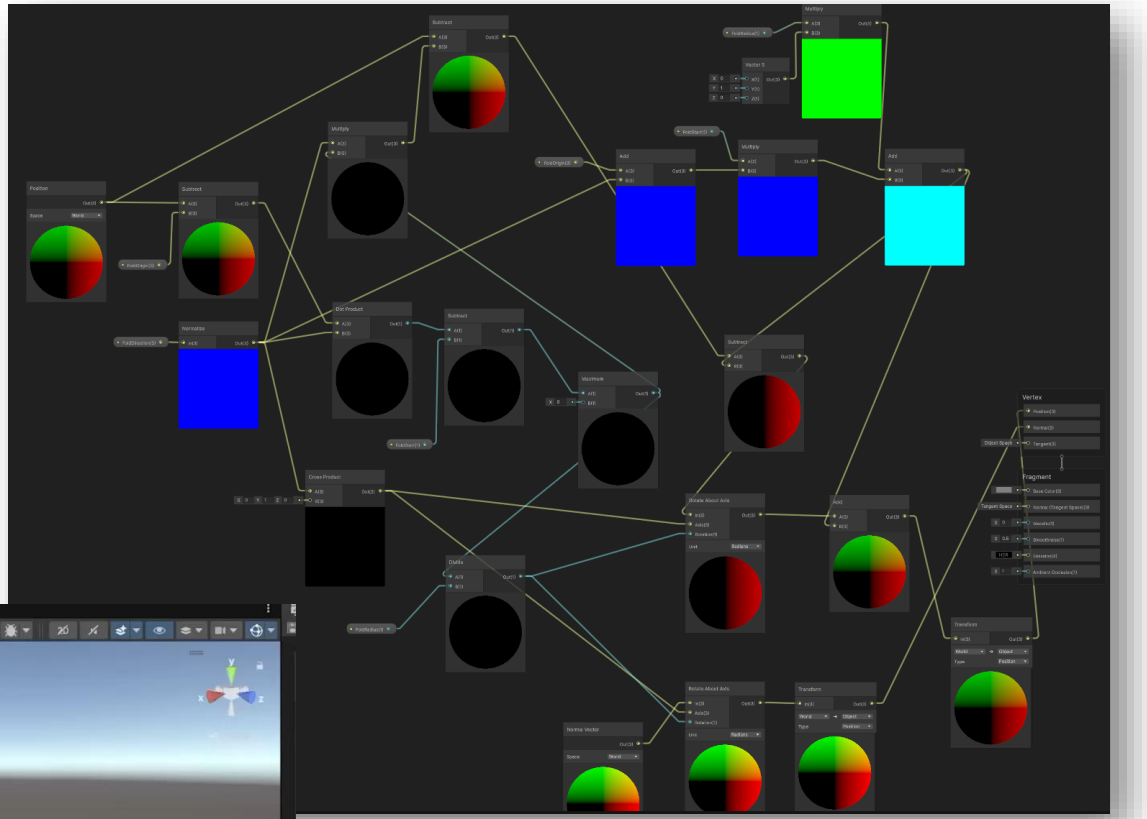
The first part of the clip is meant to demonstrate depth of field; the camera is given a large aperture (f/2.0), resulting in a shallow depth of field and supposedly thus high background blurring. The camera is first focused on the near globe, before snapping focus to the far globe behind it. In practice this difference was much smaller than expected, being most visible if you focus on the near globe as it's losing focus. Possibly something else in my configuration was poor, like the unit scale or a missing camera parameter, but it seems like additional blurring may be necessary.

The second part of the clip demonstrates adaptive camera lighting as walls and a roof appear and block the sun from illuminating the room of globes. The room is quickly consumed by darkness, but gradually everything becomes brighter and more visible, in cooler and ess saturated colours than when directly lit.

The default import material had very poor light interaction. Changing the object's base material To M_Ms_Base and re-attaching the basecolor texture greatly improved their interaction with global illumination (show before and after?)



Exercise 5: The Raster Rendering Pipeline + Shader Tuning



World bend shader.

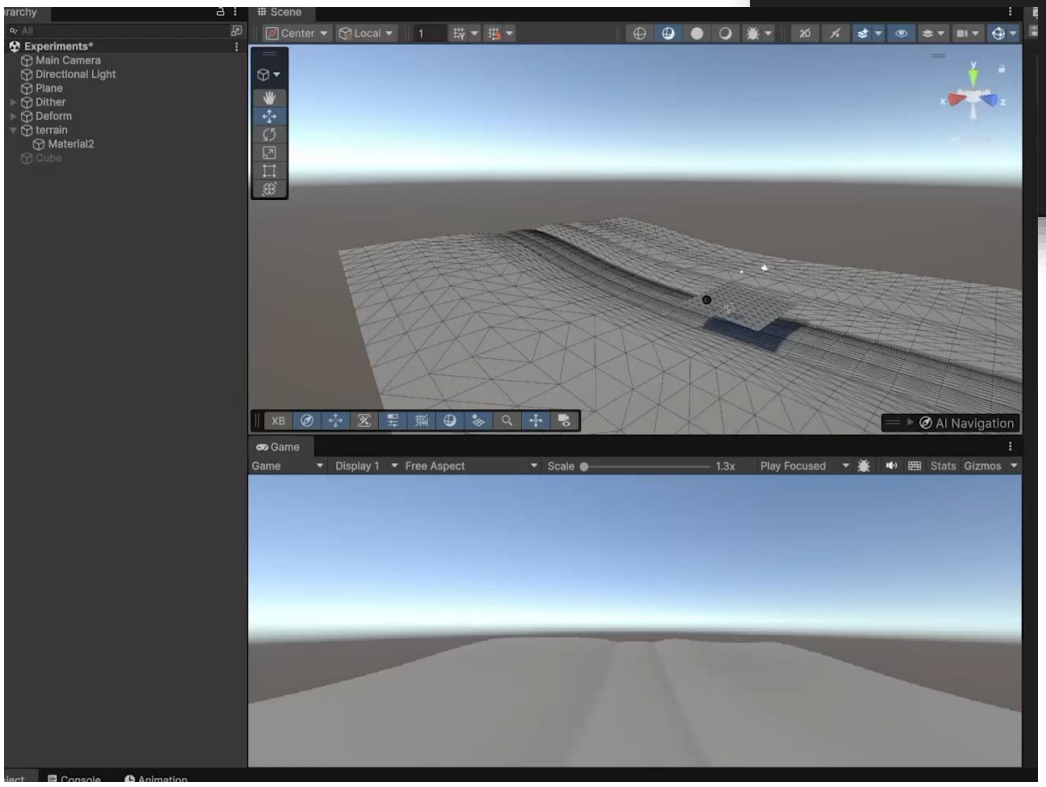
BendOrigin Vector3

BendDirection Vector3

BendStart Float

BendRadius Float

For each vertex, calculate its distance from BendOrigin in BendDirection. If above BendStart, create a normalized t coordinate and displace it around a pivot with simple geometry

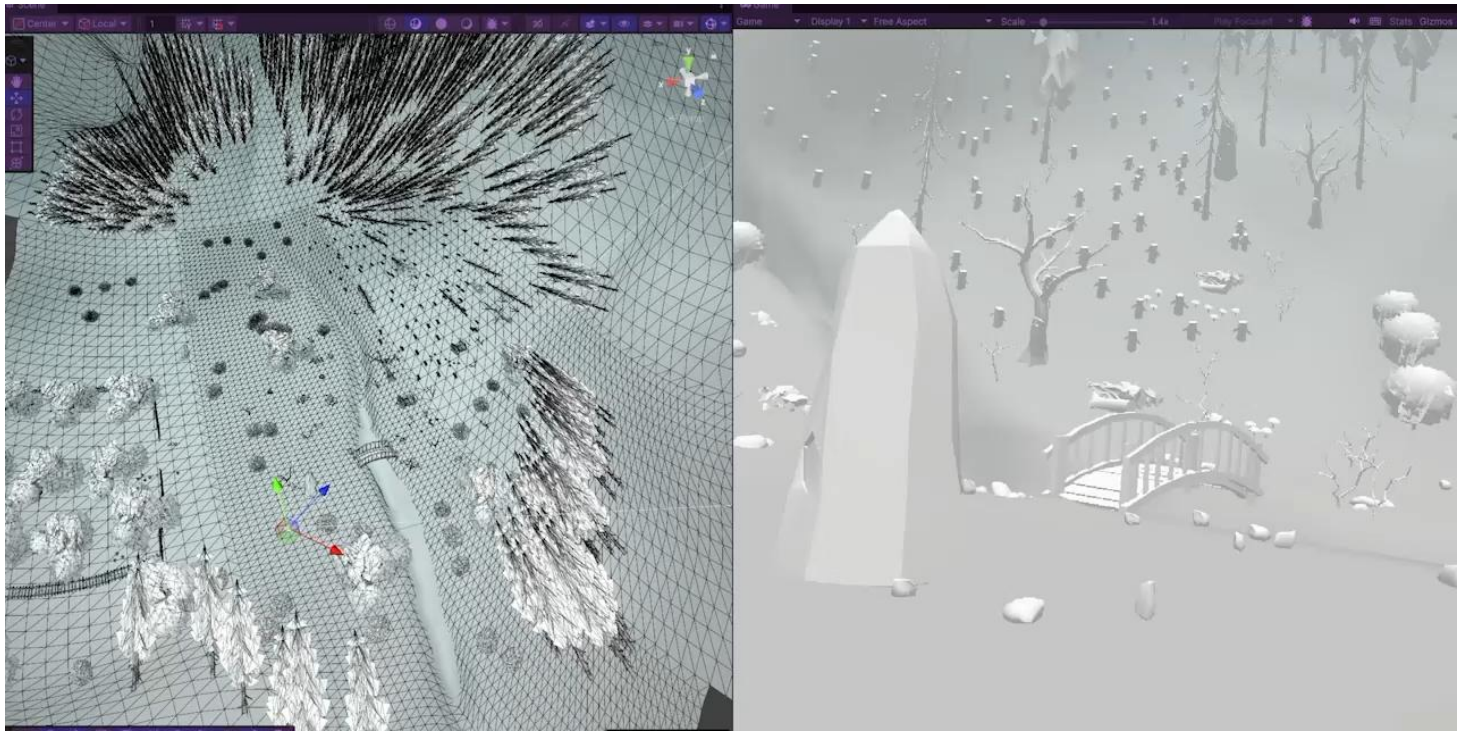


Exercise 6: Shader Programming Crash Course

Does support colour and base texture

Limitations:

- No lighting
- Requires good meshes (big vertices don't bend prettily)
- View culling is still based on world-space position. This is solvable by extending the renderers' localbounds, ideally by translating them with a C# implementation of the bend math from the shader. Distance based culling should probably also be implemented to smoothen the top edges of the bend.



Exercise 7: Real Time Graphics Optimization

Exercise 8: Real-Time Animation

Exercise 9: Tiny Virtual Production

Exercise 10: Analysis of 3D Objects